

Full Name: Nguyen Van Duy Manh

Student Code: 29211100024

Class: CMU-CS 462 DIS

TABLE OF CONTENTS

1. Building a Measurement Plan using the GQM Model.....	2
a. GQM Definition	2
b. Justification	2
2. Debating Software Size (LOC vs. Function Points.....	2
a. Arguments Against LOC	2
b. Calculation	3
3. Early Estimation with Object Points (OP)	3
4. Software Quality Theory	3
a. Quality Differences in Context	3
b. Metrics for Quality in Use	4
5. Testing Performance Metrics.....	4
a. Planning Phase	4
b. Execution Report.....	5

1. Building a Measurement Plan using the GQM Model

a. GQM Definition

- **Goal:**

- Purpose: Improve
- Issue: Satisfaction (Reliability and Usability)
- Object: MedCare software system
- Viewpoint: Doctors and patients

- **Questions and Metrics:**

- **Question 1 (Reliability):** How often does the MedCare system experience downtime during active clinic hours?

➔ **Metric:** Mean Time Between Failures (MTBF).

- **Question 2 (Usability - Efficiency):** How much time does it take for a nurse or doctor to complete a standard medical record entry?

➔ **Metric:** Average Task Completion Time.

- **Question 3 (Usability - Accuracy):** How many operational mistakes are made by clinic staff when scheduling patient appointments?

➔ **Metric:** User Error Rate.

b. Justification

- **Mean Time Between Failures:** Directly measures the extreme stability requirement. It tracks how long the system can run continuously without crashing. If the system crashes frequently, doctors are blocked from treating patients.
- **Task Completion Time & User Error Rate:** Selected to measure system usability. Since the clinic staff are busy, a truly user-friendly UI means they can enter medical records quickly and without clicking the wrong buttons

2. Debating Software Size (LOC vs. Function Points)

a. Arguments Against LOC

- **Language Dependency:** LOC counts vary drastically between programming languages. A modern framework used for the MedCare Medical Record feature

might require significantly fewer LOC than older languages to achieve the exact same functionality, unfairly reducing the calculated value.

- **Estimation Infeasibility:** LOC cannot be accurately estimated from the current requirements document. Function Points can be extracted directly from the system specifications (Inputs, Outputs, Inquiries) before a single line of code is written.
- **Lack of Business Value:** LOC measures developer typing effort, not what the clinic receives. Clinic stakeholders care about functional value (how many forms or reports the system handles), which FP measures directly.

b. Calculation

- **Unadjusted Function Point Count (UFC)**

$$\text{UFC} = (60 \times 4) + (40 \times 5) + (30 \times 4) + (5 \times 10) + (4 \times 7) = 638$$

- **Value Adjustment Factor (VAF)**

$$\text{VAF} = 0.65 + (0.01 \times \text{TDI}) = 0.65 + (0.01 \times 42) = 1.07$$

- **Final Adjusted Function Points**

$$\text{FP} = \text{UFC} \times \text{VAF} = 638 \times 1.07 = 682.66$$

3. Early Estimation with Object Points (OP)

$$\text{OP} = (15 \times 1) + (5 \times 2) + (4 \times 5) + (2 \times 10) = 65$$

$$\text{OP}_{\text{new}} = \text{OP} \times \frac{100 - r}{100} = 65 \times \frac{100 - 20}{100} = 52$$

$$\text{Effort} = \frac{\text{OP}_{\text{new}}}{\text{PROD}} = \frac{52}{13} = 4 \text{ persons / month}$$

4. Software Quality Theory

a. Quality Differences in Context

- **Internal Quality:** Evaluated statically without executing the final environment. MedCare achieved 0 internal defect density, meaning the developer's source code is structurally correct and passes isolated unit tests.

- **External Quality:** Evaluated dynamically when the software runs. The system processing too slowly is an External Quality failure (Performance Efficiency), even if the internal code logic is technically correct.
- **Quality in Use:** The extent to which the software meets end-user needs in their specific context. The receptionists making operational mistakes due to a confusing interface is a direct Quality in Use failure, as the system fails to provide effectiveness and efficiency in the real clinic setting.

b. Metrics for Quality in Use

- **To measure the receptionists' issues:**
 - **Task Error Rate:** The number of incorrect clicks or input mistakes made per appointment scheduled.
 - **Task Time:** The total time in seconds required for a receptionist to successfully complete the end-to-end appointment scheduling workflow.

5. Testing Performance Metrics

a. Planning Phase

- **Cost Limit Calculation:**
 - Budget = 200000 x 10% = 20000\$
 - Cost per testcase = 50\$
 - Max testcase by Cost = 20000 / 50 = 400\$
- **Time Limit Calculation:**
 - Total QA Hours = 3 staff x 3 weeks x 40 hours/week = 360 hours
 - Time per testcase = 2 hours
 - Max testcase by Time = 360 / 2 = 180
- **Finalized Target:** 180 Test Cases

Reasoning: Time is the primary bottleneck. Although the budget allows for 400 test cases, the QA team only possesses the physical hours to execute 180 test cases within the strict 3-week deadline.

b. Execution Report

- **Test Execution Rate:**

- Execution Rate = $\frac{\text{Executed}}{\text{Target}} = \frac{270}{180} = 150\%$

- Meaning: The QA team executed 150% of the planned workload, likely due to overtime or testing process optimizations surpassing the initial time estimations.

- **Pass Rate:**

- Pass Rate = $\frac{\text{Passed}}{\text{Executed}} = \frac{216}{270} = 0.8 = 80\%$

- Meaning: 80% of the tested scenarios functioned correctly without errors.

- **Defect Density:**

- Defect Density = $\frac{\text{Defects}}{\text{Size}} = \frac{45}{50} = 0.9 \text{ defects / KLOC}$

- Meaning: There is nearly 1 defect for every 1000 lines of code, serving as a baseline indicator of the system's current stability.